

DESARROLLO DE SOFTWARE

UNIDAD N° 5: Entorno moderno
de Desarrollo - React





Contenidos

- NodeJS
- Vite
- React
- Componentes (props y estado)
- JSX
- Renderizado (DOM - Virtual DOM)
- Eventos
- Ciclo de vida de un componente - useEffect
- TailwindCSS
- React-router-dom
- React-hook-form
- Axios
- Generación de bundler



Recursos de lectura

- <https://developer.mozilla.org/es/>
- <https://es.react.dev/>
- <https://www.reactjs.wiki/>
- <https://vite.dev/guide/>
- <https://react-hook-form.com/get-started>
- <https://tailwindcss.com/docs/installation/using-vite>



Material de Estudio

Primera Parte

Obligatorio:

- [Vite](#)
- [package.json - dependencies](#)
- [package.json - scripts](#)
- [React - component](#)
- [React - JSX](#)
- [React - props](#)
- [React - children](#)
- [React - useEffect](#)
- [React - Hooks Parte 1](#)
- [React - Hooks Parte 2](#)
- [React Hook Form - useForm](#)
- [React Hook Form - register/validaciones](#)
- [Screaming Architecture](#)



Material de Estudio

Segunda Parte

Obligatorio:

- [Atomic Design](#)
- [Tailwind css](#)
- [React Router Dom](#)



Nodejs

- Es un runtime de Javascript que funciona fuera del navegador.
- Permite ejecutar JS en la terminal y usar herramientas modernas de desarrollo.
- En frontend, no se usa como backend, es usado como entorno para instalar librerías y ejecutar scripts.

```
node -v # muestra versión de Node
```



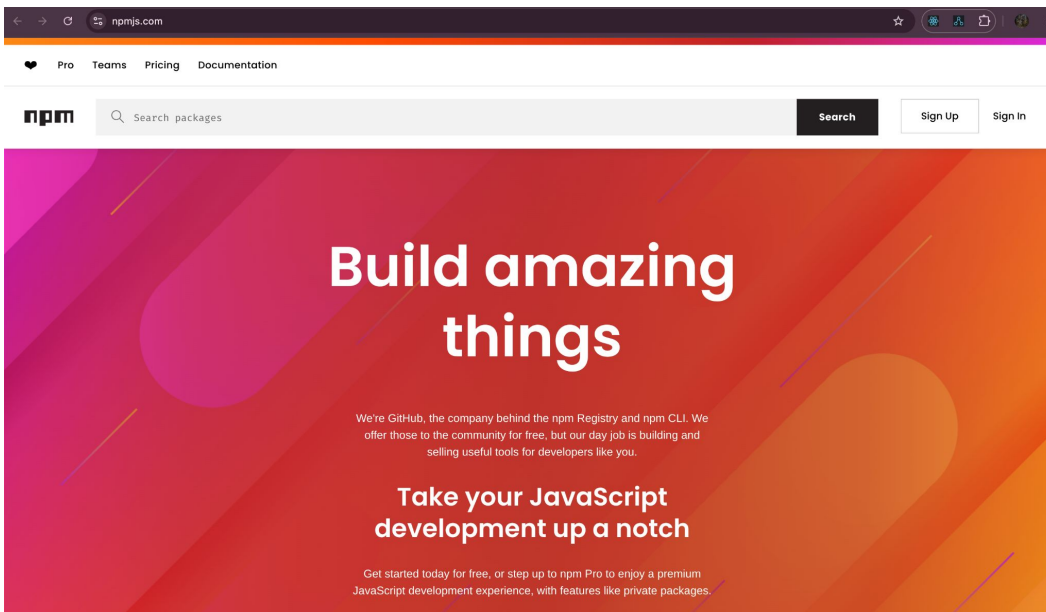
npm, yarn, pnpm

- Son gestores de dependencias
- Permite instalar librerías
- Trabajan sobre el archivo package.json

```
npm -v      # muestra versión de npm
```



Dónde están los paquetes/librerías de terceros?



<https://www.npmjs.com/>



Vite

- Es un bundler moderno y rápido para proyectos de frontend.
- Funciona como:
 - Servidor de desarrollo (npm run dev)
 - Empaquetador para producción (npm run build)
- Es uno de los bundler más rápidos que existen.
- Configuraciones mínimas.
- Soporta ESM modules.



Vite - Cómo nos ayuda?

- Gestionará librerías de terceros.
- Trabaja como servidor web.
- Puedes configurar un proxy.
- Ejecutar tests.
- Generar bundler para desplegar en servidores.



Vite - Cómo crear un proyecto?

- npm create vite@latest
- Si pide instalar algo extra, suele pedirlo la primera vez, ingresar “y” en la consola
- Ingresar nombre de proyecto
- Seleccionar framework
- Seleccionar variante
- Seleccionar instalar con npm
- Esperar....

```
> node_modules
> public
> src
eslint.config.js
vite.config.js
package-lock.json
package.json
README.md
index.html
```



Vite - Qué contiene?

- **node_modules/**: Carpeta donde se instalan todas las dependencias (librerías y paquetes) que el proyecto necesita. **NO** se versiona.
- **public/**: Archivos estáticos que no pasan por el proceso de compilación.
- **src/**: Acá va el **código fuente** de tu aplicación React.
- **eslint.config.js**: Configuración de **ESLint**, una herramienta que revisa tu código y marca errores de estilo o malas prácticas.
- **vite.config.js**: Configuración de **Vite**, el bundler que compila y levanta el servidor de desarrollo.
- **package.json**: Archivo central del proyecto. Contiene:
 - El nombre y versión del proyecto
 - Scripts de ejecución (npm run dev, npm run build, etc.)
 - Dependencias (React, Vite, etc.)
- **package-lock.json**: Versión “congelada” de las dependencias, asegura que todos los que instalen el proyecto tengan exactamente las mismas versiones.
- **index.html**: Punto de entrada de la aplicación. Aunque React controla la interfaz, este archivo define el div donde se montará la app (`<div id="root"></div>`).

```
> node_modules
> public
> src
  eslint.config.js
  vite.config.js
  package-lock.json
  package.json
  README.md
  index.html
```



package.json

- **scripts:** Comandos que se pueden ejecutar con “npm run...”
- **dependencies:** Paquetes necesarios para que la aplicación funcione. Serán parte del bundler final.
- **devDependencies:** Paquetes necesarios solamente para desarrollo.

NOTA: Hay más secciones/configuraciones que se pueden agregar en este archivo

```
{
  "name": "unidad-5",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.1.1",
    "react-dom": "^19.1.1"
  },
  "devDependencies": {
    "@eslint/js": "^9.36.0",
    "@types/react": "^19.1.13",
    "@types/react-dom": "^19.1.9",
    "@vitejs/plugin-react": "^5.0.3",
    "eslint": "^9.36.0",
    "eslint-plugin-react-hooks": "^5.2.0",
    "eslint-plugin-react-refresh": "^0.4.20",
    "globals": "^16.4.0",
    "vite": "^7.1.7"
  }
}
```



Versiones de librerías

Se sigue el estándar semver (Semantic Versioning)

MAJOR.MINOR.PATCH

- **MAJOR:** cambios grandes o incompatibles
- **MINOR:** nuevas funcionalidades, compatible
- **PATCH:** correcciones de errores



Versiones de librerías

Notación	Ejemplo	Significado	Versión que instalará
Exacta	"1.2.3"	Solo esa versión exacta.	1.2.3
Circunflejo ^	"^1.2.3"	Actualiza MINOR y PATCH , pero no cambia MAJOR .	$\geq 1.2.3 < 2.0.0$
Tilde ~	"~1.2.3"	Actualiza solo PATCH dentro del mismo MINOR.	$\geq 1.2.3 < 1.3.0$
Asterisco *	"*"	Cualquier versión.	Última disponible
Mayor que >	">1.2.3"	Acepta versiones más nuevas que la indicada.	$> 1.2.3$
Menor que <	"<2.0.0"	Solo versiones anteriores a la indicada.	$< 2.0.0$
Rango	" $\geq 1.2.3 < 2.0.0$ "	Define un rango específico de versiones válidas.	Ej: 1.2.3, 1.5.0, 1.9.9
Última (latest)	"latest"	Instala la última versión publicada en npm.	Última disponible
Pre-release	"1.2.3-beta.1"	Versión preliminar de prueba (beta, alpha, rc).	1.2.3-beta.1



React - Componente

- Es una función o clase que retorna elementos visuales.
- Los componentes permiten dividir aplicaciones en **bloques reutilizables**.

```
function Saludo() {  
  return <h1>Hola Mundo desde React</h1>;  
}  
  
export default Saludo;
```

```
import Saludo from "./Saludo";  
  
function App() {  
  return (  
    <div>  
      <Saludo />  
    </div>  
  );  
}  
  
export default App;
```




JSX

- Extensión de la sintaxis de Javascript.
- Permite escribir código similar a HTML dentro de archivos Javascript.
- Facilita la creación de contenido dinámico.

```
function Saludo() {  
  return <h1>Hola Mundo desde React</h1>;  
}  
  
export default Saludo;
```



JSX vs HTML

HTML

```
<div class="caja">  
  <h1>Hola Mundo</h1>  
</div>
```

JSX

```
const nombre = "Mundo";  
return (  
  <div className="caja">  
    <h1>Hola {nombre}</h1>  
  </div>  
)
```



JSX vs HTML

Característica	HTML	JSX (JavaScript XML)
Qué es	Lenguaje de marcado estándar usado en la web.	Extensión de JavaScript que permite escribir código parecido a HTML dentro de JS.
Dónde se usa	Archivos .html servidos por el navegador.	Archivos .jsx o .tsx dentro de proyectos React.
Sintaxis	Pura estructura de etiquetas.	Mezcla de etiquetas y expresiones JS dentro de {}.
Atributos	Palabras reservadas de HTML (ej: class, for).	Versión camelCase o modificada (ej: className, htmlFor).
Lógica de programación	No admite lógica directamente, se combina con JS aparte.	Permite incrustar variables y expresiones directamente ({} dentro del JSX).
Validación	Más flexible, el navegador interpreta errores menores.	Más estricto: cada etiqueta debe cerrarse correctamente (,).
Transformación	Se interpreta tal cual por el navegador.	Es compilado a React.createElement() antes de ejecutarse.

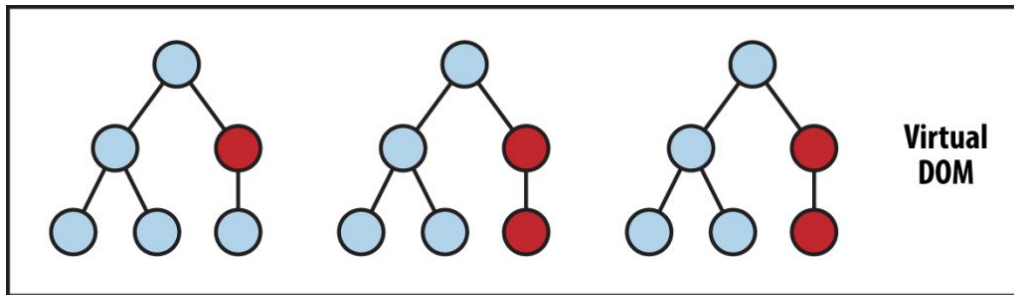


Virtual DOM

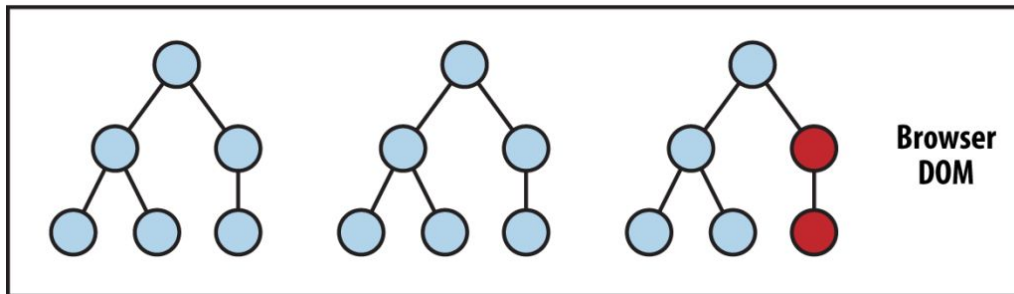
- Representación en memoria del **DOM** real.
- **NO** es una API, es un concepto, patrón utilizado por varias librerías y frameworks.
- React usa el Virtual DOM para crear y calcular en la interfaz de usuario



Virtual DOM



State Change → Compute Diff → Re-render





Virtual DOM

Característica

DOM (Document Object Model)

Virtual DOM (React)

Qué es

Representación del documento HTML como un árbol de nodos.

Una copia ligera del DOM real, mantenida en memoria por React.

Ubicación

Vive en el navegador.

Vive en la memoria de la aplicación React.

Actualizaciones

Directas: cada cambio en un nodo fuerza al navegador a recalcular estilos, redibujar y re-renderizar partes de la página.

Indirectas: React actualiza el Virtual DOM, luego compara (diff) con el DOM real y aplica solo los cambios necesarios.

Rendimiento

Menos eficiente: si hay muchos cambios seguidos, cada actualización puede ser costosa.

Más eficiente: minimiza las operaciones en el DOM real, que son las más pesadas.

Uso típico

JS puro, jQuery, manipulación directa de elementos HTML.

React y frameworks modernos que buscan optimizar la actualización de la UI.

Ejemplo de cambio

```
document.getElementById("titulo").textContent = "Hola";
```

```
setState("Hola") → React actualiza el Virtual DOM, detecta el cambio y actualiza solo el nodo titulo en el DOM real.
```



React - props

- Las props es la forma de pasar datos desde un padre a un hijo.
- Son inmutables. Es decir los hijos no modifican los valores de las props, solo el padre.

```
function Saludo(props) {  
  return <h1>Hola {props.nombre}!</h1>;  
}  
  
export default function App() {  
  return (  
    <div>  
      <Saludo nombre="Ana" />  
      <Saludo nombre="Luis" />  
    </div>  
  );  
}
```



React - children

- Prop especial que se pasa a un componente. Es un objeto que contiene los elementos que envuelve un componente.

```
function Card(props) {  
  return (  
    <div className="card">  
      <h2>{props.title}</h2>  
      <div>{props.children}</div>  
    </div>  
  )  
}
```

```
<Card title="Título de la tarjeta">  
  <p>Contenido de la tarjeta</p>  
</Card>
```




React - state

- Representa el estado interno de un componente o hook.
- Almacena datos, y en base a estos el componente puede cambiar de apariencia o comportamiento.
- Se usa el hook useState para definir un state en el componente.

```
import { useState } from "react";

function Contador() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Contador: {count}</h1>
      <button onClick={() => setCount(count + 1)}>Sumar</button>
      <button onClick={() => setCount(count - 1)}>Restar</button>
    </div>
  );
}

export default Contador;
```



React - props vs state

- Las *props* son un objeto que se pasan como argumentos de un componente padre a un componente hijo. Son inmutables y no se pueden modificar desde el componente hijo.
- El *state* es un valor que se define dentro de un componente. Su valor es inmutable (no se puede modificar directamente) pero se puede establecer un valor nuevo del estado para que React vuelva a renderizar el componente.



React - Renderizado condicional

- Se puede renderizar un JSX según condición:
 - Operador ternario `condición ? A : B`
 - Operador lógico &&

```
function Login({ isLoggedIn }) {  
  return (  
    <div>  
      {isLoggedIn ? <h1>Bienvenido!</h1> : <h1>Por favor inicia sesión</h1>}  
    </div>  
  );  
}
```

```
function Mensaje({ mostrar }) {  
  return <div>{mostrar && <p>Este es un mensaje secreto 🗝️</p>}</div>;  
}
```



React - Listas usando .map

- Para renderizar varios elementos de un array se usa .map()
- **IMPORTANTE:** cada elemento debe tener una propiedad key única para ayudar a React a identificar cambios

```
function ListaProductos() {  
  const productos = [  
    { id: 1, nombre: "Laptop", precio: 1200 },  
    { id: 2, nombre: "Mouse", precio: 25 },  
    { id: 3, nombre: "Teclado", precio: 45 },  
  ];  
  
  return (  
    <ul>  
      {productos.map((p) => (  
        <li key={p.id}>  
          {p.nombre} - ${p.precio}  
        </li>  
      ))}  
    </ul>  
  );  
}
```



Ejercicio #25 - Mostrar/ocultar un listado basado productos

- Crear un proyecto desde cero con “npm create vite@latest”, configurarlo como se indicó en diapositivas anteriores.
- Con Visual Studio Code, abrir la carpeta del proyecto.
- Abrir el archivo App.jsx.
- Crear un array de productos:

```
const productos = [  
  { id: 1, nombre: "Laptop", precio: 1200 },  
  { id: 2, nombre: "Mouse", precio: 25 },  
  { id: 3, nombre: "Teclado", precio: 45 },  
];
```

- Definir un state llamado “show”, como valor como defecto será false.
- Borrar contenido de la función return del componente App.
- Agregar un botón, con la prop onClick, al hacer click se debe cambiar el valor del state “show”. El contenido del botón cambia basado en el valor del state “show”.
- Debajo del botón, se debe agregar, de forma condicional basado en la variable “show”, el listado de productos.

Resultado: el listado de productos se muestra o no, dependiendo de cómo el usuario interactúa con el botón



React - eventos

- Los eventos son similares a JS, pero se escriben en **camelCase** y se pasan como funciones.
- React implementa un **sistema de eventos sintéticos** que funciona de manera uniforme en todos los navegadores.
- Un **evento sintético** en React es una abstracción de los eventos nativos del navegador. Está diseñado para proporcionar una interfaz consistente entre navegadores para la gestión de eventos en aplicaciones React.

```
function Boton() {  
  const handleClick = () => {  
    alert("Hiciste clic!");  
  };  
  
  return <button onClick={handleClick}>Click aquí</button>;  
}
```



React - comunicación entre componentes

- A) Padre -> Hijo
- El Padre pasa los datos al Hijo mediante props.

```
function Hijo({ mensaje }) {  
  return <h2>{mensaje}</h2>;  
}  
  
function Padre() {  
  return <Hijo mensaje="Hola desde el padre 🖐️" />;  
}
```



React - comunicación entre componentes

B) Hijo -> Padre

- El Padre pasa una función al Hijo como prop, y el Hijo lo ejecuta.
- Esto permite que el Hijo comunique eventos o datos hacia el Padre

```
function Hijo({ onSaludar }) {  
  return (  
    <button onClick={() => onSaludar("Hola desde el hijo!")}>  
      Saludar  
    </button>  
  );  
}  
  
function Padre() {  
  const manejarSaludo = (mensaje) => {  
    alert(mensaje);  
  };  
  
  return <Hijo onSaludar={manejarSaludo} />;  
}
```




Ejercicio #26 - Mostrar/ocultar un listado basado productos (Padre - Hijo)

- Tomar como base el Ejercicio #25
- Crear un nuevo componente llamado List.
- El componente List se encargará de dibujar la lista de productos, la cual es pasada desde el componente App como prop.
- Crear un componente Button genérico, que reciba el evento onClick.
- Estos dos nuevos componentes deben ser en archivos separados.

Resultado: mismo comportamiento que el Ejercicio #25



React - useEffect

Permite ejecutar efectos secundarios en componentes funcionales.

Un **efecto secundario** es cualquier operación fuera del renderizado puro, por ejemplo:

- Llamadas a una API.
- Suscripciones.
- Manipulación de eventos.
- Interacciones con el localStorage.

Recibe dos parámetros:

- La función que se ejecutará al cambiar las dependencias o al renderizar el componente.
- Un array de dependencias. Si cambia el valor de alguna dependencia, ejecutará la función.



React - useEffect

```
import { useEffect, useState } from 'react'

function Counter() {
  const [count, setCount] = useState(0)

  useEffect(() => {
    console.log('El contador se ha actualizado')
  }, [count])

  return (
    <>
      <p>Contador: {count}</p>
      <button onClick={() => setCount(count + 1)}>Aumentar</button>
    </>
  )
}
```



React - useEffect

Segundo parámetro → un arreglo de dependencias:

- [] → solo en el **montaje** (cuando el componente se renderiza por primera vez).
- [variable] → cada vez que variable cambie.
- *Sin segundo parámetro* → en cada render.



React - useEffect

Segundo parámetro → un arreglo de dependencias:

- [] → solo en el **montaje** (cuando el componente se renderiza por primera vez).
- [variable] → cada vez que variable cambie.
- *Sin segundo parámetro* → en cada render.



React - Ciclo de vida de un componente

- **Montaje (Mount):** Cuando renderiza el componente por primera vez.
- **Actualización (Update):** Cuando el componente se renderiza por cambio en una props o state
- **Desmontar (Unmount):** Cuando el componente se desmonta del árbol de React.



React - Ciclo de vida de un componente

```
useEffect(() => {  
  // Mount  
  console.log('Componente montado');  
  
  return () => {  
    // Unmount  
    console.log('Componente desmontado');  
  };  
}, []);
```

```
useEffect(() => {  
  console.log('El contador ha sido actualizado:', count);  
}, [count]);
```



React - Ciclo de vida de un componente

Fase del ciclo	Cómo se maneja con useEffect	Ejemplo
Montaje	<code>useEffect(() => {...}, [])</code>	Llamar API al cargar
Actualización	<code>useEffect(() => {...}, [dependencia])</code>	Reaccionar a cambios de estado/props
Desmontaje	<code>useEffect(() => { return () => {...} }, [])</code>	Limpiar timers o eventos



React - Simulación de fetch

- Simular el fetch de datos en un useEffect
- Cuando se esté realizando el “fetch”, se debe mostrar un mensaje al usuario: “Buscando datos”
- Cuando el “fetch” termine, mostrar los datos obtenidos.



React - Hooks

- Son **funciones especiales** de React que permiten usar **estado** y otras características sin necesidad de escribir componentes de clase.
- **Reglas básicas de los Hooks:**
 - Solo se llaman en el **nivel superior** del componente (no dentro de if o loops).
 - Solo se usan en **componentes funcionales** o en otros **Custom Hooks**.
- Hooks es gancho y, precisamente, lo que hacen, es que te permiten enganchar tus componentes funcionales a todas las características que ofrece React.



React - Hooks

Hook	Uso principal	Ejemplo
useState	Manejo de estado local.	<pre>const [count, setCount] = useState(0);</pre>
useEffect	Efectos secundarios (fetch, timers, subscripciones).	<pre>useEffect(() => {...}, []);</pre>
useContext	Acceder a valores globales con Context API.	<pre>const user = useContext(UserContext);</pre>
useRef	Referenciar elementos o valores mutables sin disparar render.	<pre>const inputRef = useRef(null);</pre>
useReducer	Estado complejo con múltiples acciones (alternativa a useState).	<pre>const [state, dispatch] = useReducer(reducer, initState);</pre>
useMemo	Memorizar valores calculados.	<pre>const total = useMemo(() => calc(items), [items]);</pre>
useCallback	Memorizar funciones para evitar renders innecesarios.	<pre>const handleClick = useCallback(() => {...}, []);</pre>



React - Custom Hooks

- Son **funciones de JavaScript** que **usan Hooks de React** y encapsulan lógica reutilizable.
- Su nombre **debe empezar con use** (ej: useFetch, useForm).
- Permiten separar lógica de negocio de la UI.



React - Custom Hooks

```
import { useState } from "react";

function useCounter(initialValue = 0) {
  const [count, setCount] = useState(initialValue);

  const increment = () => setCount((c) => c + 1);
  const decrement = () => setCount((c) => c - 1);
  const reset = () => setCount(initialValue);

  return { count, increment, decrement, reset };
}

export default useCounter;
```



React - Custom Hooks

```
import { useState } from "react";

function useCounter(initialValue = 0) {
  const [count, setCount] = useState(initialValue);

  const increment = () => setCount((c) => c + 1);
  const decrement = () => setCount((c) => c - 1);
  const reset = () => setCount(initialValue);

  return { count, increment, decrement, reset };
}

export default useCounter;
```

```
import React from "react";
import useCounter from "./useCounter";

export default function CounterApp() {
  const { count, increment, decrement, reset } = useCounter(5);

  return (
    <div>
      <h1>Contador: {count}</h1>
      <button onClick={increment}>+</button>
      <button onClick={decrement}>-</button>
      <button onClick={reset}><img alt="refresh icon" data-bbox="725 645 740 665"/></button>
    </div>
  );
}
```



React - Formularios

Controlados (useState)

No controlados (useRef)

Quién maneja el valor

React (estado)

El DOM (input real)

Facilidad para validar

✓ Muy fácil (el valor ya está en el estado)

✗ Más trabajo (hay que leer manualmente el valor)

Re-renderizado

Ocurre en cada cambio

No ocurre en cada cambio

Uso típico

Formularios complejos, validación en tiempo real

Formularios simples, cuando solo se necesita el valor al enviar



React - Inputs Controlados

```
import React, { useState } from "react";

export default function InputControladoConError() {
  const [nombre, setNombre] = useState("");
  const [error, setError] = useState("");

  const handleChange = (e) => {
    const value = e.target.value;
    setNombre(value);

    if (value.length < 3) {
      setError("El nombre debe tener al menos 3 caracteres");
    } else {
      setError("");
    }
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!error && nombre) {
      console.log("Nombre:", nombre);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>Nombre:</label>
      <input value={nombre} onChange={handleChange} />
      {error && <p style={{ color: "red" }}>{error}</p>}
      <button type="submit">Enviar</button>
    </form>
  );
}
```




React - Inputs No Controlados

```
import React, { useRef, useState } from "react";

export default function InputNoControladoConError() {
  const emailRef = useRef();
  const [error, setError] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    const email = emailRef.current.value;

    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
      setError("El email no es válido");
    } else {
      setError("");
      console.log("Email:", email);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>Email:</label>
      <input type="email" ref={emailRef} />
      {error && <p style={{ color: "red" }}>{error}</p>}
      <button type="submit">Enviar</button>
    </form>
  );
}
```



React - Inputs No Controlados

```
import React, { useRef, useState } from "react";

export default function InputNoControladoConError() {
  const emailRef = useRef();
  const [error, setError] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    const email = emailRef.current.value;

    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
      setError("El email no es válido");
    } else {
      setError("");
      console.log("Email:", email);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>Email:</label>
      <input type="email" ref={emailRef} />
      {error && <p style={{ color: "red" }}>{error}</p>}
      <button type="submit">Enviar</button>
    </form>
  );
}
```



Ejercicio #27 - Formulario y validaciones

- Requisitos funcionales:
 - Crear la página Login con los requisitos:
 - username - requerido
 - password - requerido
 - Se debe mostrar mensaje de error de validación, debajo de cada input, en color rojo.
 - Si los datos son correctos, mostrarlo por consola
- Requisitos de diseño de app:
 - Usar inputs controlados
 - Debe existir un componente Input. El cual se encargará de dibujar el label, input y error en caso de existir.



React - React-hook-form

Librería de basado en React que permite:

- Gestionar estado interno de un form.
- Manipular componentes Controlados y No Controlados.
- Gestionar validaciones.
- Gestionar formularios complejos

Instalación usando línea de comandos:

- `npm install react-hook-form`



React - React-hook-form: useForm

Es un hook que nos permitirá inicializar un formulario, y nos proveerá un conjunto de objetos y funciones para poder gestionar el cambio de valores en el formulario, errores, validaciones, etc.

```
import { useForm } from 'react-hook-form';
import './App.css';

function App() {
  const {
    register,
    handleSubmit,
  } = useForm();

  const onSubmit = (data) => {
    console.log(data);
  };

  return (
    <>
      <form onSubmit={handleSubmit(onSubmit)} >
        <input {...register('name')} />
        <input {...register('password')} />
        <button type='submit'>Enviar</button>
      </form>
    </>
  );
}

export default App;
```



React - React-hook-form: useForm

- **register(name, options?):** Conecta un input al formulario. name = clave del campo (puede ser con path para objetos, ej. user.email). options = validaciones (ej: { required: true, minLength: 3 }). Trabaja sobre elementos No Controlados.
- **handleSubmit(onValid, onInvalid?):** Envuelve la función onSubmit. Valida el formulario y llama onValid(data) si todo OK, o onInvalid(errors) si hay errores.

```
import { useForm } from 'react-hook-form';
import './App.css';

function App() {
  const {
    register,
    handleSubmit,
  } = useForm();

  const onSubmit = (data) => {
    console.log(data);
  };

  return (
    <>
      <form onSubmit={handleSubmit(onSubmit)} >
        <input {...register('name')} />
        <input {...register('password')} />
        <button type='submit'>Enviar</button>
      </form>
    </>
  );
}

export default App;
```



React - React-hook-form: validaciones básicas

required	El campo es obligatorio.	<code>{ required: "Este campo es obligatorio" }</code>	"Este campo es obligatorio"
minLength	Longitud mínima de caracteres.	<code>{ minLength: { value: 3, message: "Mínimo 3 caracteres" } }</code>	"Mínimo 3 caracteres"
maxLength	Longitud máxima de caracteres.	<code>{ maxLength: { value: 10, message: "Máximo 10 caracteres" } }</code>	"Máximo 10 caracteres"
pattern	Expresión regular que debe cumplirse.	<code>{ pattern: { value: /^[A-Za-z]+\$/, message: "Solo letras" } }</code>	"Solo letras"
min	Valor mínimo (para números).	<code>{ min: { value: 18, message: "Debe ser mayor o igual a 18" } }</code>	"Debe ser mayor o igual a 18"
max	Valor máximo (para números).	<code>{ max: { value: 99, message: "Debe ser menor o igual a 99" } }</code>	"Debe ser menor o igual a 99"
validate	Validación personalizada (función).	<code>{ validate: value => value === "ok" }</code>	



React - React-hook-form: validaciones básicas

```
<form onSubmit={handleSubmit(onSubmit)} className="flex flex-col gap-2">
  <input
    placeholder="Nombre"
    {...register('nombre', {
      required: 'El nombre es obligatorio',
      minLength: { value: 3, message: 'Mínimo 3 caracteres' },
    })}
  />
  {errors.nombre && <span>{errors.nombre.message}</span>}

  <input
    type="email"
    placeholder="Correo"
    {...register('email', {
      pattern: { value: /^[^@ ]+@[^@ ]+\.[^@ .]{2,}$/,
        message: 'Correo inválido',
      },
    })}
  />
  {errors.email && <span>{errors.email.message}</span>}

  <input
    type="number"
    placeholder="Edad"
    {...register('edad', {
      min: { value: 18, message: 'Debe ser mayor de edad' },
    })}
  />
  {errors.edad && <span>{errors.edad.message}</span>}

  <button type="submit">Enviar</button>
</form>
```




React - React-hook-form: errores

```
const { formState: { errors } } = useForm();
```

```
<input
  placeholder="Nombre"
  {...register('nombre', {
    required: 'El nombre es obligatorio',
    minLength: { value: 3, message: 'Mínimo 3 caracteres' },
  })}
/>
{errors.nombre} && <span>{errors.nombre.message}</span>}
```



Ejercicio #28 - Formulario y validaciones

- Migrar ejercicio #27 a react-hook-form



Screaming Architecture

- Buscamos que al leer simplemente las carpetas comprendamos de qué se trata cada módulo.
- Buscamos que sea simple, no desordenado y fácil de encontrar cada cosa.
- No organizaremos en capas técnicas.
- Organizaremos basado en funcionalidades mediante módulos.



Screaming Architecture

1. por tipo

```
└─ src/
  └─ assets/
  └─ api/
  └─ configs/
  └─ components/
    └─ SignUpForm.tsx
    └─ Employees.tsx
    └─ PaymentForm.tsx
    └─ Button.tsx
  └─ hooks/
    └─ usePayment.ts
    └─ useUpdateEmployee.ts
    └─ useEmployees.ts
    └─ useAuth.tsx
  └─ lib/
  └─ services/
  └─ states/
  └─ utils/
```

2. por tipo y features

```
└─ src/
  └─ assets/
  └─ api/
  └─ configs/
  └─ components/
    └─ auth/
      └─ SignUpForm.tsx
    └─ payment/
      └─ PaymentForm.tsx
    └─ common/
      └─ Button.tsx
    └─ employees/
      └─ EmployeeList.tsx
      └─ EmployeeSummary.tsx
  └─ hooks/
    └─ auth/
      └─ useAuth.ts
    └─ payment/
      └─ usePayment.ts
    └─ employees/
      └─ useEmployees.ts
      └─ useUpdateEmployee.ts
  └─ lib/
  └─ services/
  └─ states/
  └─ utils/
```



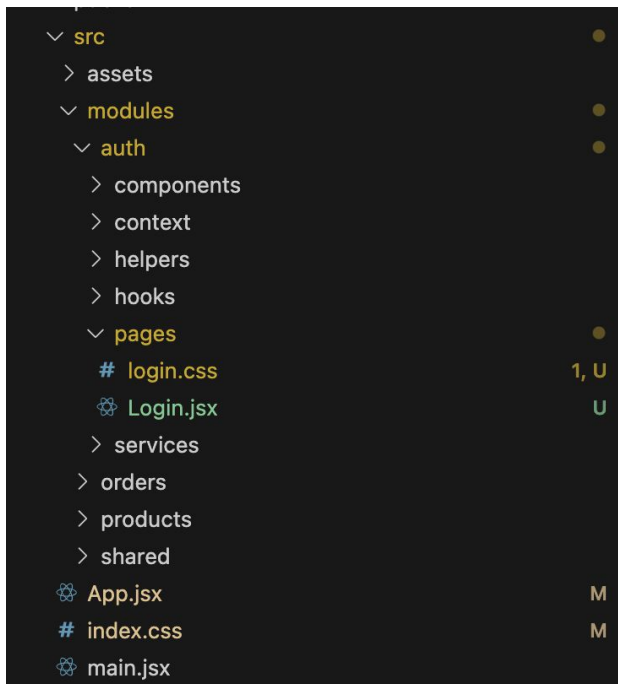
Screaming Architecture

3. screaming architecture

```
└─ src/
  └─ assets/
  └─ modules/
    └─ core/
      └─ components/
      └─ design-system/
        └─ Button.tsx
      └─ hooks/
      └─ lib/
      └─ utils/
    └─ payment/
      └─ components/
        └─ PaymentForm.tsx
      └─ hooks/
        └─ usePayment.ts
      └─ lib/
      └─ services/
      └─ states/
      └─ utils/
    └─ auth/
      └─ components/
        └─ SignUpForm.tsx
      └─ hooks/
        └─ useAuth.ts
      └─ lib/
      └─ services/
      └─ states/
      └─ utils/
    └─ employees/
      └─ components/
        └─ EmployeeList.tsx
        └─ EmployeeSummary.tsx
      └─ hooks/
        └─ useEmployees.ts
```



Screaming Architecture en nuestro proyecto





Ejercicio #29 - Formulario y validaciones

- Migrar ejercicio #28 a la nueva organización de carpetas.



Atomic Design

El **Atomic Design** es una **metodología para crear interfaces de usuario (UI)** de forma **modular y escalable**, propuesta por **Brad Frost**.

La idea se basa en cómo la **química organiza la materia**:

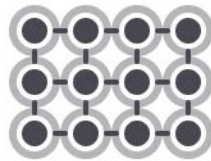
átomos → moléculas → organismos → plantillas → páginas.



Átomos



Moléculas



Organismos



Plantillas



Páginas



Atomic Design

Nivel	Descripción	Ejemplo práctico (React)
Átomos (Atoms)	Son los elementos más pequeños e indivisibles de la UI. No tienen dependencia de otros componentes.	Botón, Input, Label, Icono
Moléculas (Molecules)	Combinación de átomos que trabajan juntos para cumplir una función simple.	Input + Label + ErrorMessage → InputField
Organismos (Organisms)	Conjunto de moléculas y/o átomos que forman secciones completas de una interfaz.	Navbar, Card de producto, Formulario de login
Plantillas (Templates)	Estructuras de página que definen el layout, pero aún sin datos reales.	Estructura de “Página de login” con Header, Form y Footer
Páginas (Pages)	Instancias finales de las plantillas con datos reales.	LoginPage, ProductsPage, ProfilePage



Atomic Design

Dashboard

Cerrar Sesión

- Átomo: button, span
- Molécula: Button (componente personalizado), Title
- Organismo: Header



Atomic Design

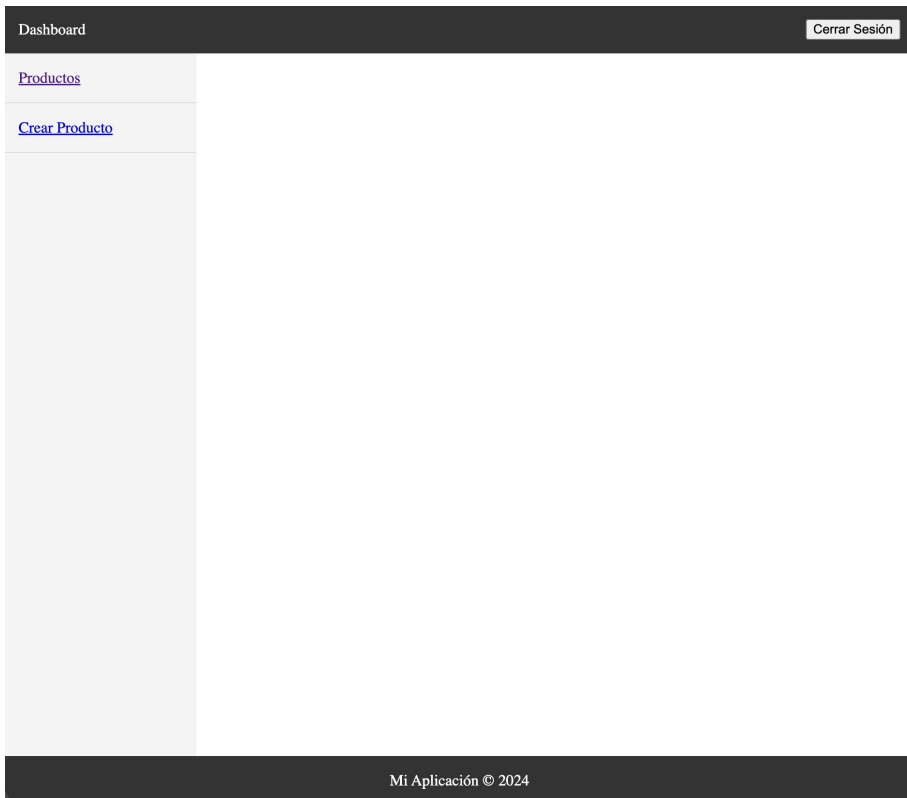
[Productos](#)

[Crear Producto](#)

- Átomo: nav, ul, li, a
- Molécula: Menú seleccionable
- Organismo: Sidebar



Atomic Design



- Template: Esqueleto del dashboard que define el diseño base

Atomic Design

Dashboard

Cerrar Sesión

[Productos](#)
[Crear Producto](#)

Productos

Buscar productos...

Codigo	Nombre	Stock	Vencimiento
A001	Leche	25	2025-10-15
B002	Arroz	100	--
A003	Yogur	15	2025-09-30
A004	Fideos	50	--
A005	Atún en lata	40	2027-01-01
A006	Galletitas	0	2025-12-20

Mi Aplicación © 2024

- Página: Componente ProductsPage.jsx. El componente toma el Template, e implementa la página visualizando el contenido de la página. En este caso listado de productos.

Atomic Design

Dashboard

Cerrar Sesión

[Productos](#)
[Crear Producto](#)

Crear un Producto

SKU:

Codigo Unico:

Nombre:

Descripcion:

Precio:

Stock:

Guardar

Mi Aplicación © 2024

- Página: Componente CreateProductPage.jsx. El componente toma el Template, e implementa la página visualizando el contenido de la página. En este caso creación de producto.



Ejercicio #30 - Formulario y validaciones

- Basado en el ejercicio #29:
 - Crear los componentes necesarios y ubicarlos en las carpetas correspondientes.
 - Pensar en qué componentes pueden ser reutilizables.
 - Si se encuentra componentes que pueden ser reutilizables, crear un componente personalizado y promoverlo al nivel que corresponde



Tailwind css

Funciona escaneando todos sus archivos HTML, componentes JavaScript y cualquier otra plantilla en busca de nombres de clase, generando los estilos correspondientes y luego escribiendolos en un archivo CSS estático.



Tailwind css

- Instalación:
 - npm install tailwindcss
 - npm install -D @tailwindcss/vite
 - En [vite.config.js](https://vite.dev/config/) agregar "import tailwindcss from '@tailwindcss/vite'" y en plugins agregar "tailwindcss()"

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';

// https://vite.dev/config/
export default defineConfig({
  plugins: [
    react(),
    tailwindcss(),
  ],
});
```

- En index.css en la primera línea agregar '@import "tailwindcss";'

```
@import "tailwindcss";

body {
  margin: 0;
  height: 100dvh;
}
```



Tailwind css

- Instalar extensión en VSC



Tailwind CSS IntelliSense

Tailwind Labs  tailwindcss.com



11,620,888



(115)

Intelligent Tailwind CSS tooling for VS Code

Disable



Uninstall



Auto Update





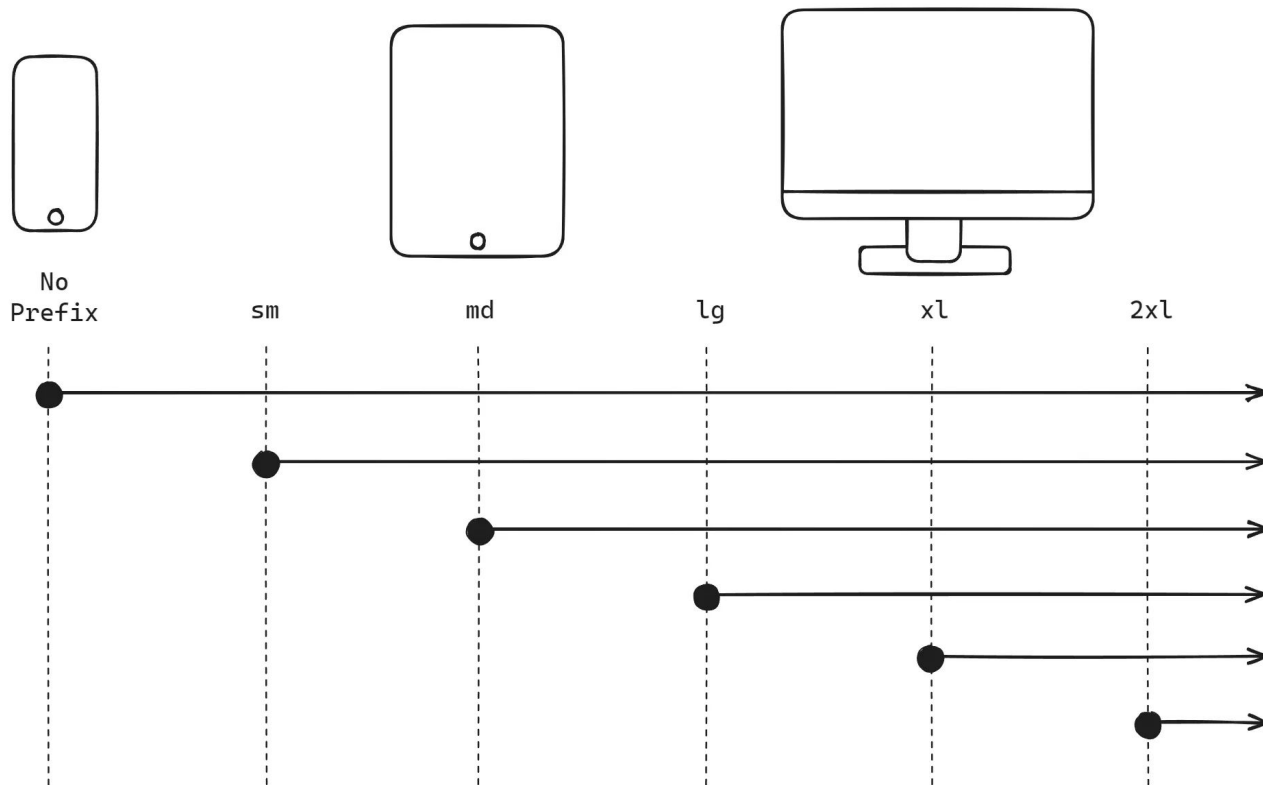
Tailwind css - Responsive

- Tailwind css se basa en el principio mobile-first
- Los breakpoints se usan solamente para aplicar $\geq$ a determinada resolución.

Breakpoint prefix	Minimum width	CSS
<code>'sm'</code>	40rem (640px)	<code>'@media (width >= 40rem) { ... }'</code>
<code>'md'</code>	48rem (768px)	<code>'@media (width >= 48rem) { ... }'</code>
<code>'lg'</code>	64rem (1024px)	<code>'@media (width >= 64rem) { ... }'</code>
<code>'xl'</code>	80rem (1280px)	<code>'@media (width >= 80rem) { ... }'</code>
<code>'2xl'</code>	96rem (1536px)	<code>'@media (width >= 96rem) { ... }'</code>



Tailwind css - Responsive





Ejercicio #31 - Tailwind css

- Basado en el ejercicio #30:
 - Replicar el diseño de la página de Login de la Unidad 4.
 - Respetar el principio mobile-first



Vite - configurar proxy

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';

// https://vite.dev/config/
export default defineConfig({
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:5001/',
      },
    },
  },
  plugins: [
    react(),
    tailwindcss(),
  ],
});
```



Fetch

```
const response = await fetch('api/auth/login', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify(data),
});

if (!response.ok) {
  const errorData = await response.json();

  setErrorMessage(frontendErrorMessage[errorData.code]);

  return;
}

setErrorMessage('');

const token = await response.json();
```

```
const frontendErrorMessage = {
  1000: 'Usuario y/o Contraseña no son correctos',
};

export {
  frontendErrorMessage,
};
```



Ejercicio #32 - Conectar al endpoint login

- Basado en el ejercicio #31:
 - Configurar lo necesario en vite.
 - Llamar al endpoint de login, si falla, controlar el error y mostrar mensajes al usuario basado en code.



React Router Dom

Biblioteca que nos facilita el enrutamiento en aplicaciones React de forma eficiente y sencillo. Su principal responsabilidad es facilitar la navegación.

Instalación:

```
npm install react-router-dom
```



React Router Dom - Definir rutas

Se crea un array, cada objeto del array tiene como atributos mínimamente “path” y “element”.

- **path:** representa la url de la página. Puede contener parámetros dinámicos.
- **element:** componente React que se montará cuando el usuario entre a la url definida en path

React Router Dom - Definir rutas

```
const routes = createBrowserRouter([
  {
    path: '/',
    element: <App />,
    errorElement: <ErrorNotFoundPage />,
    children: [
      {
        path: '/',
        element: <MainPage />
      },
      {
        path: 'careers',
        element: <CareersPage />
      },
      {
        path: 'careers/:id',
        element: <CareerDetail />
      }
    ]
  }
]);
```

```
<RouterProvider router={ routes } />
```



Cómo realizamos la navegación?

Con etiqueta “a”

```
<nav className='app-layout__sidebar'>
  <h1>Menú</h1>
  <ul>
    <li>
      <a href='careers'>Listar Carreras</a>
    </li>
    <li>
      <a href='careers'>Listar Estudiantes</a>
    </li>
  </ul>
</nav>
```

Con component “Link”

```
<nav className='app-layout__sidebar'>
  <h1>Menú</h1>
  <ul>
    <li>
      <Link to={ 'careers' }>Listar Carreras</Link>
    </li>
    <li>
      <Link to={ 'students' }>Listar Estudiantes</Link>
    </li>
  </ul>
</nav>
```



Cómo realizamos la navegación?

Con component “NavLink”

```
<nav className='app-layout__sidebar'>
  <h1>Menú</h1>
  <ul>
    <li>
      <NavLink
        to={ 'careers' }
        className={ ({ isActive }) => isActive ? 'app-layout__menu-active' : '' }
      >
        Listar Carreras
      </NavLink>
    </li>
    <li>
      <NavLink
        to={ 'students' }
        className={ ({ isActive }) => isActive ? 'app-layout__menu-active' : '' }
      >
        Listar Estudiantes
      </NavLink>
    </li>
  </ul>
</nav>
```



Cómo realizamos la navegación internamente en un componente o hook ?

```
const navigate = useNavigate();
```

```
<button  
  onClick={ () => navigate(`/careers/${career.id}`) }  
>  
  Ver  
</button>
```



Cómo realizamos la navegación internamente en un componente o hook? (volver página anterior)

```
const navigate = useNavigate();
```

```
<button onClick={() => navigate(-1)}>Volver atras</button>
```



Ejercicio #33

- Basado en el proyecto de github https://github.com/vchibilisco/Dsw2025_Unidad5
- Crear un Componente [Dashboard.jsx](#)
- Este nuevo componente por el momento, solo tendrá un mensaje que dirá “Hola a todos”.
- La página Login estará relacionado con una url “/login”.
- Este componente estará relacionado con una url “/app”.
- Cuando el inicio de sesión termine correctamente, debemos navegar desde la url “/login” a la url “/app”



Ejercicio #34 - Layout / Dashboard

- Basado en el ejercicio #33
- Usando Tailwind css, pensando en el principio Atomic Design y Mobile First:
 - Crear los componentes necesarios para Header, Sidebar y Footer.
 - El componente Dashboard debe utilizar los componentes anteriormente mencionados aplicando el mismo Layout/Maqueta que se usó en la unidad 4.

Objetivo:

- Replicar apariencia, color y funcionamiento del Dashboard de la unidad 4.

Ayudas:

- Usar la documentación oficial de [Tailwind css](#) para verificar que clases se deben usar para replicar el comportamiento del Dashboard de la unidad 4



Rutas privadas

El objetivo es encontrar un mecanismo donde si el usuario no está autenticado sea redirigido directamente a la página login.



Rutas privadas

Suponiendo que vamos a exponer información del usuario a nivel global, se puede seguir esta implementación.

Usando API Context de React, vamos a crear un mecanismo que nos permita proteger las URL desde el módulo auth.



Rutas privadas - AuthProvider

Crear un componente AuthProvider en la carpeta “context”. Esto ayudará a exponer información al resto de la App.

```
import { createContext, useEffect, useState } from 'react';

const AuthContext = createContext();

function AuthProvider({ children }) {
  const [isAuthenticated, setIsAuthenticated] = useState(false);

  useEffect(() => {
    setIsAuthenticated(localStorage.getItem('token') ? true : false);
  }, []);

  return (
    <AuthContext.Provider value={{ isAuthenticated }} >
      { children }
    </AuthContext.Provider>
  );
};

export { AuthProvider, AuthContext };
```



Rutas privadas - useAuth

Crear un useAuth en la carpeta “hook”. Mediante este hook vamos a acceder a la información de la sesión desde cualquier lugar de la app.

```
import { useContext } from 'react';
import { AuthContext } from '../context/AuthProvider';

const useAuth = () => {
  const context = useContext(AuthContext);

  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }

  return context;
};

export default useAuth;
```



Rutas privadas - ProtectedRoute

Crear un componente ProtectedRoute. Recibe como prop el componente Page (children), y realiza la verificación si el usuario está logueado. En caso de no estarlo retorna un Navigate a /login, caso contrario retorna el componente Page.

```
import { Navigate } from 'react-router-dom';
import useAuth from '../hooks/useAuth';

function ProtectedRoute({ children }) {
  const { isAuthenticated } = useAuth();

  if (!isAuthenticated) {
    return <Navigate to='/login' />;
  }

  return children;
};

export default ProtectedRoute;
```



Rutas privadas - Implementación

- AuthProvider: engloba toda la app.
- ProtectedRoute: engloba solamente a los componentes que deben ser privados.

```
function App() {  
  const router = createBrowserRouter([  
    {  
      path: '/login',  
      element: <LoginPage />,  
    },  
    {  
      path: '/orders',  
      element: (  
        <ProtectedRoute>  
        | <ListOrdersPage />  
        </ProtectedRoute>  
      ) ,  
    },  
  ] );  
  
  return (  
    <AuthProvider>  
    | <RouterProvider router={router} />  
    </AuthProvider>  
  );  
}  
  
export default App;
```



Dependencias entre rutas

Definición de dependencias entre rutas.
Esto nos permitirá renderizar un componente dentro de otro componente basado en la URL.
En la imagen, los componentes Home, ListProductsPage y ListOrdersPage se renderizan dentro de Dashboard.

Pero para lograr eso, no solamente es necesario realizar esta dependencia. Si no indicar dentro de Dashboard donde debe ser renderizado.

```
{
  path: '/admin',
  element: (
    <ProtectedRoute>
      <Dashboard />
    </ProtectedRoute>
  ),
  children: [
    {
      path: '/admin/home',
      element: <Home />,
    },
    {
      path: '/admin/products',
      element: <ListProductsPage />,
    },
    {
      path: '/admin/orders',
      element: <ListOrdersPage />,
    },
  ],
},
```




Dependencias entre rutas

Outlet: es un componente que ayuda a renderizar las rutas hijas en caso de que le URL coincida con el componente.

```
import { Outlet } from 'react-router-dom';
```

```
<main  
  className="  
    p-5  
  "  
>  
  <Outlet />  
</main>
```